

Laboratorio n°3

Felipe Vargas

202360541-7

Repositorio: <https://github.com/fvargasusm/ICN292-Lab3-Vargas-Felipe>

Resumen ejecutivo:

El presente informe detalla la construcción en n8n de un producto mínimo viable que automatiza el triage de las solicitudes de devolución del canal e-commerce de AndesHogar SpA, proceso que hoy se resuelve a mano en un buzón compartido con un tiempo promedio de primera respuesta de tres días hábiles frente a las veinticuatro horas que promete la política publicada. Se construyeron tres workflows: uno de triage que recibe las solicitudes por un Webhook en POST, las clasifica con un nodo Switch en modo Rules que aplica las reglas de negocio en el orden exacto de la política, escribe la decisión y su motivo en cada rama, consulta el valor de la UF del día a la API pública de mindicador.cl, deja constancia de cada caso en una planilla de Google y responde al solicitante; uno emisor que envía las cargas de prueba por POST a ese webhook; y uno programado con Schedule Trigger que lee el registro, consolida el día con el nodo Summarize y emite un único mensaje de resumen. Con la semilla personal $S = 135$ los parámetros resultantes son $U = \$65.000$ y $D = 28$ días, y con ellos se procesaron las quince solicitudes entregadas: cinco se aprobaron de inmediato, tres se rechazaron y siete quedaron en revisión, por un monto total de \$1.251.820 y una tasa de aprobación automática de 33,3%. Las tres solicitudes de control se comportaron como correspondía con independencia de la semilla, lo que confirma que los parámetros fueron bien calculados. El flujo también se sometió a entradas inválidas y a casos de frontera: una solicitud con monto exactamente igual a U y otra con días exactamente iguales a D quedaron ambas dentro de lo aceptado, tal como exige la regla de comparaciones inclusivas, y tres entradas defectuosas fueron capturadas por la regla de datos inválidos con motivos diferenciados según la causal. Se identificaron además dos casos que el flujo no puede resolver: una solicitud con el monto escrito en palabras, que hace fallar el nodo Switch con el mensaje cannot convert to number, y una con un SKU inexistente, que atraviesa toda la validación y se clasifica como si fuera legítima porque el flujo nunca consulta el catálogo de productos. Una prueba adicional dejó en evidencia un límite de capacidad: al enviar las quince solicitudes con cuatrocientos milisegundos de separación, cuatro de ellas superaron el tiempo de espera por congestión de ejecuciones concurrentes, y solo al espaciarlas a cuatro segundos el lote completo se procesó sin fallas. Se concluye que el MVP debe adoptarse como sistema definitivo del triage, porque resuelve el cuello de botella real del proceso y lo demostró sobre datos reales, pero que antes de operar en producción deben cerrarse tres brechas concretas que las propias pruebas dejaron a la vista: la validación contra el catálogo, el manejo de volumen en ráfaga y la visibilidad de las reglas de negocio para quienes no editan el flujo. Toda la evidencia que permite verificar lo anterior está publicada en el repositorio indicado arriba: los tres workflows exportados en formato json, la copia del registro producido por el flujo y las capturas de las ejecuciones citadas en cada sección.

Parte-A: Flujo de triage

Semilla personal: $S = 135$.

Parametros resultantes: $U = 30.000 + 1.000 \times (135 \bmod 50) = 30.000 + 1.000 \times 35 = \65.000 ; $D = 7 + 7 \times (135 \bmod 4) = 7 + 7 \times 3 = 28$ días.

Estructura del workflow

El flujo de triage quedó compuesto por doce nodos. El primero es un Webhook configurado en método POST con el path devoluciones-135 y la respuesta delegada al nodo final, de modo que el solicitante recibe la decisión y no un simple acuse de recibo. El segundo es el nodo Switch en modo Rules, donde vive toda la política. De él salen siete conexiones que llegan a cuatro nodos Edit Fields, uno por cada ruta posible, que escriben los campos ruta y motivo. Las cuatro ramas convergen en un nodo NoOp llamado Consolidar rutas, cuya función es dar un punto de referencia único al que puedan apuntar las expresiones posteriores. Después viene un HTTP Request en GET a la API pública <https://mindicador.cl/api>, de cuya respuesta se consume el campo uf.valor, que corresponde al valor de la UF del día en pesos y permite expresar el monto de la solicitud en unidades de fomento. Luego el nodo de registro arma la fila de trazabilidad, el nodo de Google Sheets la escribe en la planilla, un HTTP Request en POST representa la notificación al cliente y el Respond to Webhook cierra la ejecución devolviendo el registro completo.

El nodo Consolidar rutas merece una explicación porque no aparece en el material de la ayudantía. El HTTP Request que consulta la UF reemplaza el item que recibe, de modo que después de consultarlo ya no están disponibles el monto ni los demás campos de la solicitud, y hay que recuperarlos nombrando el nodo anterior. Con cuatro ramas distintas no existe un único nodo anterior al que apuntar, por lo que se agregó este punto de convergencia para que todas las expresiones posteriores puedan escribirse de la misma forma sin importar por qué ruta haya pasado la solicitud.

Configuración del Switch

Las reglas se escribieron en el orden de la política, cada condición en su propia regla, con Send Data to All Matching Outputs desmarcada y el Fallback Output renombrado como APROBACION. Los valores numéricos corresponden a la semilla personal declarada arriba.

#	Condicion	Detalle	Salida
1	String · is empty	{{ \$json.body.id_solicitud }}	INVALIDO_CAMPO
2	Number · ≤ 0	{{ \$json.body.monto.toNumber() }}	INVALIDO_MONTO
3	Number · > 28	{{ \$json.body.dias_desde_compra.toNumber() }}	RECHAZO_PLAZO
4	String · equal to	estado_producto = danado_por_uso	RECHAZO_ESTADO
5	Number · > 65000	{{ \$json.body.monto.toNumber() }}	REVISION_MONTO
6	String · equal to	estado_producto = con_fallas	REVISION_FALLA
—	Fallback Output	—	APROBACION

El orden no es un detalle de redacción. DEV-1120 y DEV-1126 cumplen simultáneamente la regla del monto y la del estado con fallas, y ambas causales conducen a REVISION, de modo que la ruta final no cambia; lo que si cambia es el motivo que queda registrado. Como la regla 5 se evalúa antes que la 6, el registro dice que la solicitud fue derivada por superar el umbral. Si el orden fuera el inverso, esas mismas solicitudes quedarían archivadas con la justificación de una falla del producto, y una auditoría posterior leería un proceso distinto del que realmente ocurrió. Lo mismo sucede con DEV-C03, que excede el plazo y además supera el umbral: la regla del plazo se evalúa primero, de modo que se rechaza en lugar de derivarse a revisión, que es lo correcto porque una solicitud fuera de plazo ya esta resuelta y mandarla a revisión implicaría ocupar tiempo de bodega en un caso que la política ya cerró.

Una decisión propia fue como escribir el campo motivo. El enunciado indica cuatro nodos Edit Fields, uno por ruta, pero la regla 7 exige que cada registro tenga un motivo y en tres de esas cuatro ramas desembocan dos causales distintas. Un texto fijo por rama haría que las dos causales quedaran indistinguibles en el registro. La solución adoptada fue escribir el motivo con una expresión condicional dentro de cada Edit Fields, que pregunta por la misma causal que el Switch evaluó primero y devuelve el texto correspondiente. De esta manera se conserva la estructura de cuatro nodos que pide el enunciado y al mismo tiempo el registro distingue si el rechazo fue por plazo o por estado, y si la revisión fue por monto o por falla.

Resultados de las quince solicitudes

Las quince solicitudes se enviaron desde el workflow emisor por POST a la URL de producción del webhook. Los resultados fueron los siguientes:

ID	Monto	Días	Estado	Ruta	Motivo
DEV-1115	19.990	3	sin_abrir	APROBACION	Cumple plazo, estado y monto
DEV-1116	69.980	5	abierto_sin_uso	REVISION	Monto 69980 supera el umbral U de 65000
DEV-1117	24.990	12	con_fallas	REVISION	Producto declarado con fallas
DEV-1118	79.990	4	sin_abrir	REVISION	Monto 79990 supera el umbral U de 65000
DEV-1119	45.990	25	danado_por_uso	RECHAZO	Producto danado por uso
DEV-1120	89.990	2	con_fallas	REVISION	Monto 89990 supera el umbral U de 65000
DEV-1121	289.990	9	sin_abrir	REVISION	Monto 289990 supera el umbral U de 65000
DEV-1122	45.990	2	sin_abrir	APROBACION	Cumple plazo, estado y monto
DEV-1123	39.980	15	danado_por_uso	RECHAZO	Producto danado por uso
DEV-1124	49.980	1	abierto_sin_uso	APROBACION	Cumple plazo, estado y monto
DEV-1125	34.990	22	sin_abrir	APROBACION	Cumple plazo, estado y monto
DEV-1126	79.990	8	con_fallas	REVISION	Monto 79990 supera el umbral U de 65000
DEV-C01	9.990	1	sin_abrir	APROBACION	Cumple plazo, estado y monto
DEV-C02	289.990	2	sin_abrir	REVISION	Monto 289990 supera el umbral U de 65000
DEV-C03	79.990	29	abierto_sin_uso	RECHAZO	Fuera de plazo: 29 días, el máximo D es 28

El balance es de cinco aprobaciones por \$160.940, tres rechazos por \$165.960 y siete revisiones por \$924.920, con un monto total de \$1.251.820. Ninguna solicitud cayó en la ruta de datos inválidos, lo que era esperable porque el archivo entregado no contiene defectos de forma.

Las tres solicitudes de control confirman que los parámetros están bien calculados. DEV-C01, de \$9.990, se aprueba con cualquier umbral posible porque el mínimo valor que puede tomar U es \$30.000. DEV-C02, de \$289.990, se deriva a revisión con cualquier umbral porque el máximo que puede tomar U es \$79.000. Y DEV-C03, con 29 días, se rechaza con cualquier plazo porque el máximo que puede tomar D es 28 días, que es precisamente el valor correspondiente a esta semilla. Las tres se comportaron como estaba previsto.

Triage (Switch Rules)

INPUT

Webhook devoluciones 1 item

```
{
  "headers": {
    "host": "pipevargask.app.n8n.cloud",
    "user-agent": "n8n",
    "content-length": "172",
    "accept": "application/json,text/html,application/xhtml+xml,application/xml,text/*;q=0.9, image/*;q=0.8, */*;q=0.7",
    "accept-encoding": "gzip, br",
    "cdn-loop": "cloudflare; loops=1; subreqs=1",
    "cf-connecting-ip": "20.52.126.7",
    "cf-eu-via": "15",
    "cf-ipcountry": "DE",
    "cf-ray": "a3f3f8f68f2ae529-TXL",
    "cf-visitor": "{\\\"scheme\\\":\\\"https\\\"}",
    "cf-worker": "n8n.cloud",
    "content-type": "application/json",
    "x-forwarded-for": "20.52.126.7, 104.23.199.188",
    "x-forwarded-host": "pipevargask.app.n8n.cloud",
    "x-forwarded-port": "443",
    "x-forwarded-proto": "https",
    "x-forwarded-server": "traefik-prod-users-gwc-26-59cd5b855d-64n1r",
    "x-is-trusted": "yes",
    "x-real-ip": "20.52.126.7"
  },
  "params": {
  },
  "query": {
  },
  "body": {
    "id_solicitud": "DEV-C03",
    "sku": "MW-150",
    "unidades": "1",
    "monto": "79990",
    "dias_desde_compra": "29",
    "estado_producto": "abierto_sin_uso",
    "email_cliente": "control03@example.cl"
  },
  "webhookUrl": "https://pipevargask.app.n8n.cloud/webhook/devoluciones-135",
  "executionMode": "production"
}
```

Docs ⓘ x

OUTPUT ⚠

Schema Table JSON

INVALIDO_CAMPO INVALIDO_MONTO RECHAZO_PLAZO (1 item) RECHAZO_ESTADO REVISION_MONTO REVISION_FALLA APROBACION

headers	params	query	body	webhookUrl	executionMode
host : pipevargask.app.n8n.cloud user-agent : n8n content-length : 172 accept : application/json,text/html,application/xhtml+xml,application/xml,text/*;q=0.9, image/*;q=0.8, */*;q=0.7 accept-encoding : gzip, br cdn-loop : cloudflare; loops=1; subreqs=1 cf-connecting-ip : 20.52.126.7 cf-eu-via : 15 cf-ipcountry : DE cf-ray : a3f3f8f68f2ae529-TXL cf-visitor : {\"scheme\":\"https\"} cf-worker : n8n.cloud content-type : application/json x-forwarded-for : 20.52.126.7, 104.23.199.188 x-forwarded-host : pipevargask.app.n8n.cloud x-forwarded-port : 443 x-forwarded-proto : https x-forwarded-server : traefik-prod-users-gwc-26-59cd5b855d-64n1r x-is-trusted : yes x-real-ip : 20.52.126.7	{empty object}	{empty object}	id_solicitud : DEV-C03 sku : MW-150 unidades : 1 monto : 79990 dias_desde_compra : 29 estado_producto : abierto_sin_uso email_cliente : control03@example.cl	https://pipevargask.app.n8n.cloud/webhook/devoluciones-135	production

Figura 1 — Ejecución exitosa de DEV-C03 por la URL de producción. El panel de entrada muestra los 29 días y el de salida la rama RECHAZO_PLAZO.

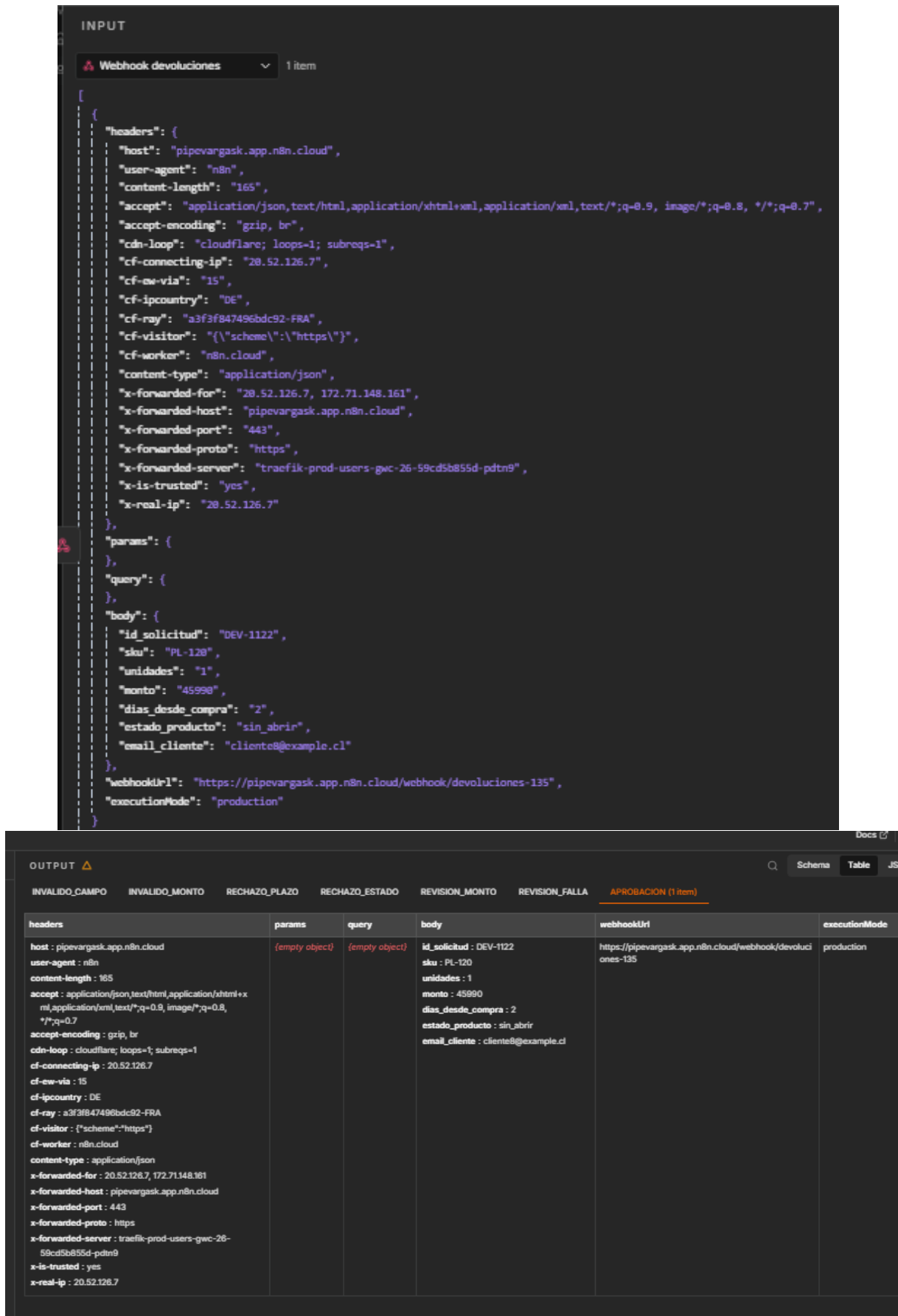


Figura 2 — Ejecución exitosa de DEV-1122. Ninguna regla se cumple y la solicitud sale por el Fallback Output renombrado APROBACION.

A1 fx id_solicitud										
	A	B	C	D	E	F	G	H	I	J
1	id_solicitud	sku	monto	dias_desde_compra	ruta	motivo	U_aplicado	D_aplicado		
2	DEV-1115	AU-210	19990	3	APROBACION	Cumple plazo, e	65000	28		
3	DEV-1116	TE-330	69980	5	REVISION	Monto 69980 su	65000	28		
4	DEV-1117	HV-050	24990	12	REVISION	Producto declar	65000	28		
5	DEV-1118	MW-150	79990	4	REVISION	Monto 79990 su	65000	28		
6	DEV-1119	PL-120	45990	25	RECHAZO	Producto danad	65000	28		
7	DEV-1120	AS-260	89990	2	REVISION	Monto 89990 su	65000	28		
8	DEV-1121	LV-330	289990	9	REVISION	Monto 289990 s	65000	28		
9	DEV-1122	PL-120	45990	2	APROBACION	Cumple plazo, e	65000	28		
10	DEV-1123	AU-210	39980	15	RECHAZO	Producto danad	65000	28		
11	DEV-1124	HV-050	49980	1	APROBACION	Cumple plazo, e	65000	28		
12	DEV-1125	TE-330	34990	22	APROBACION	Cumple plazo, e	65000	28		
13	DEV-1126	MW-150	79990	8	REVISION	Monto 79990 su	65000	28		
14	DEV-C01	CB-010	9990	1	APROBACION	Cumple plazo, e	65000	28		
15	DEV-C02	LV-330	289990	2	REVISION	Monto 289990 s	65000	28		
16	DEV-C03	MW-150	79990	29	RECHAZO	Fuera de plazo:	65000	28		
17										
18										
19										
20										

Figura 3 — Registro producido por el flujo: las quince solicitudes con su ruta, su motivo y los valores de U y D aplicados.

Parte-B: Flujo programado de resumen

El tercer workflow se compone de ocho nodos. Un Schedule Trigger lo dispara, un nodo de Google Sheets lee la planilla de registro completa, un nodo IF separa el caso en que hay solicitudes del caso en que no las hay, el nodo Summarize agrupa por ruta contando solicitudes y sumando montos, un nodo Aggregate reúne las filas resultantes en un único item, un Edit Fields arma el mensaje y un HTTP Request lo envía.

El nodo Aggregate es necesario porque el Summarize entrega una fila por cada ruta distinta, es decir tres items, y todo nodo en n8n se ejecuta una vez por cada ítem que recibe. Sin ese paso intermedio el flujo enviaría tres mensajes separados en lugar del único mensaje consolidado que pide el enunciado.

Frecuencia elegida

Se configuró una corrida diaria a las 20:00, hora de Chile. La elección responde a dos razones. La primera es que el enunciado pide consolidar el día y calcular el monto total del día y la tasa de aprobación automática, y ambas son métricas de un período cerrado: una corrida cada pocas horas entregaría tasas calculadas sobre un puñado de solicitudes, cifras que oscilan sin significar nada con volúmenes pequeños y que además no son comparables entre si. La segunda es la hora concreta: el canal e-commerce recibe solicitudes hasta la noche, de modo que un corte mas temprano dejaría las solicitudes de la tarde fuera del resumen del día en que ocurrieron y las empujaría al del día siguiente, desfasando el corte respecto del día calendario. El resumen tampoco es una herramienta de monitoreo en tiempo real sino el cierre de la jornada para Servicio al Cliente, que necesita saber cuántas devoluciones quedaron pendientes de revisión manual; eso se mira una vez, al final del día.

Resultado de la ejecución

Ejecutado sobre el registro de las quince solicitudes, el flujo entregó un total de quince solicitudes por \$1.251.820, con el detalle de APROBACION con 5 solicitudes por \$160.940, REVISION con 7 por \$924.920 y RECHAZO con 3 por \$165.960, y una tasa de aprobación automática de 33,3%. Esa cifra es la que mide directamente el valor del MVP: de cada tres solicitudes que entran, una se resuelve sola y sin intervencion humana.

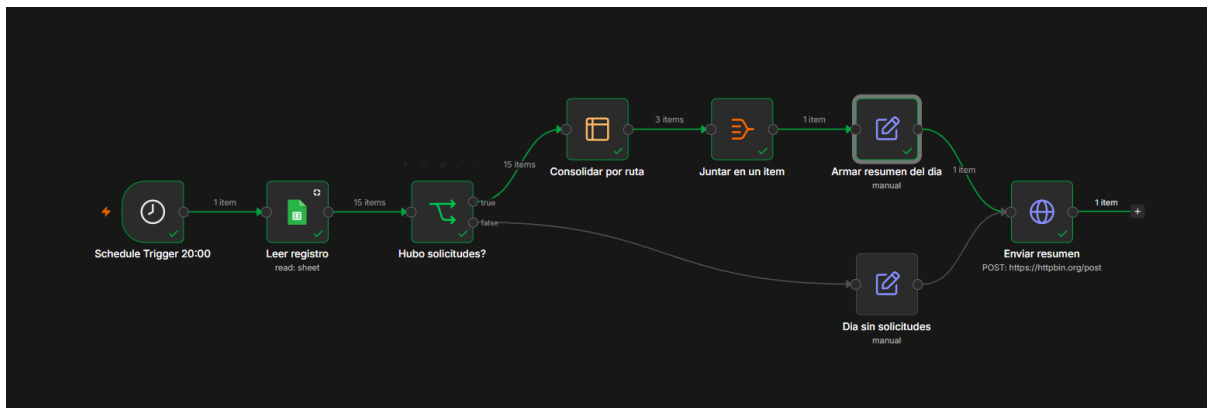


Figura 4 — Ejecución del flujo de resumen: quince filas leídas, tres tras la consolidación por ruta y un único ítem al final.

fecha	total_solicitudes	monto_total_dia	detalle_por_ruta	tasa_aprobacion_automatica	mensaje
22-09-2020	15	1251820	APROBACION: 5 solicitudes por \$804800 [RECHAZO: 7 solicitudes por \$804800] RECHAZO: 3 solicitudes por \$804800	33.3	Resumen de devoluciones del 22-09-2020. Total 15 solicitudes por \$804800. Por ruta: APROBACION: 5 solicitudes por \$804800 [RECHAZO: 7 solicitudes por \$804800] [RECHAZO: 3 solicitudes por \$804800]. Tasa de aprobación automática: 33.3%.

Figura 5 — Salida del nodo que arma el resumen, con el detalle por ruta, el monto total del día y la tasa de aprobación automática.

Qué ocurre un día sin solicitudes

Sin una previsión explícita, un día sin solicitudes es indistinguible de una falla. La lectura del registro devolvería cero ítems, cada nodo posterior se ejecutaría cero veces y la ejecución terminaría en verde sin enviar nada, que es exactamente lo mismo que se observa cuando el workflow quedó desactivado o cuando el trigger no se disparó. Nadie recibe un mensaje y nadie tiene forma de saber por qué.

Para resolverlo se activó Always Output Data en el nodo de lectura, de modo que entrega un ítem vacío cuando no hay filas, y se agregó una rama explícita que envía un mensaje informando que no se recibieron solicitudes y que el registro está vacío. La diferencia es que la ausencia de movimientos se comunica como información en lugar de como silencio. Se comprobó aplicando temporalmente un filtro que ningún registro satisface: el flujo tomó la rama falsa del IF y emitió el mensaje correspondiente.

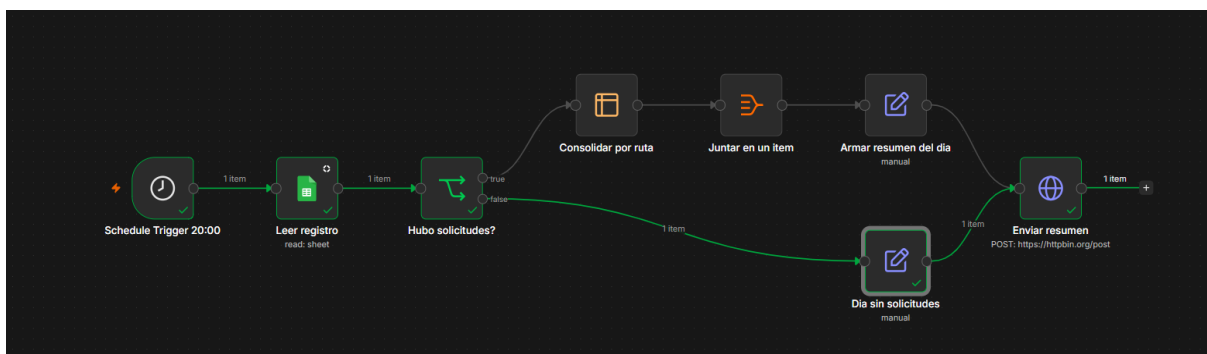


Figura 6 — Misma ejecución con el registro vacío: el IF deriva por la salida falsa y se activa la rama de día sin solicitudes.

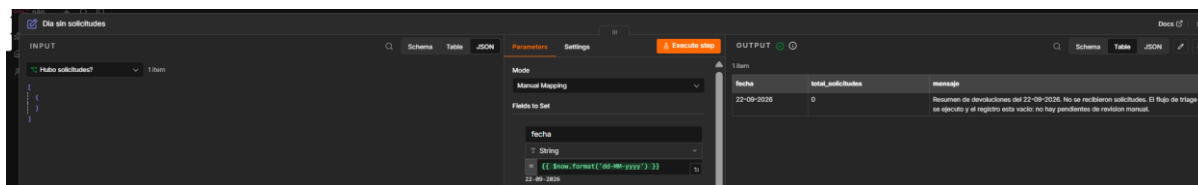


Figura 7 — Mensaje emitido cuando no hay solicitudes en el registro.

Parte-C: Errores y casos borde

1. Entradas inválidas

Se documentan cuatro entradas inválidas distintas, de las cuales la última hace fallar un nodo y no simplemente tomar la ruta de datos inválidos.

Carga enviada	Comportamiento esperado	Comportamiento observado
id_solicitud vacío, monto 25000, 3 días	Regla 1 la captura antes que cualquier otra y la clasifica como DATOS_INVALIDOS	DATOS_INVALIDOS, motivo: Falta el campo id_solicitud. El flujo termina sin error
DEV-I02, monto 0, 3 días	Regla 2: el monto no es mayor que cero, DATOS_INVALIDOS	DATOS_INVALIDOS, motivo: El monto declarado no es mayor que cero
DEV-I03, monto -15000, 3 días	Regla 2 nuevamente, por monto negativo	DATOS_INVALIDOS, mismo motivo que el caso anterior
DEV-X01, monto "veinte mil", 3 días	El nodo debe fallar: no existe conversión posible de ese texto a número	El nodo Triage (Switch Rules) falla con el mensaje cannot convert to number. El webhook responde 500 y no se clasifica nada

Las tres primeras se clasifican correctamente, pero con motivos diferenciados: la primera indica que falta el identificador y las otras dos que el monto no es mayor que cero. Esa distinción es consecuencia directa de la decisión de escribir el motivo con una expresión condicional, explicada en la Parte A. La cuarta es de naturaleza distinta: no se clasifica, sino que detiene la ejecución.

enviado	ruta_devuelta	motivo_devuelto
empty	DATOS_INVALIDOS	Falta el campo id_solicitud
DEV-I02	DATOS_INVALIDOS	El monto declarado no es mayor que cero
DEV-I03	DATOS_INVALIDOS	El monto declarado no es mayor que cero

Figura 8 — Las tres entradas inválidas clasificadas como DATOS_INVALIDOS con motivos diferenciados según la causal.

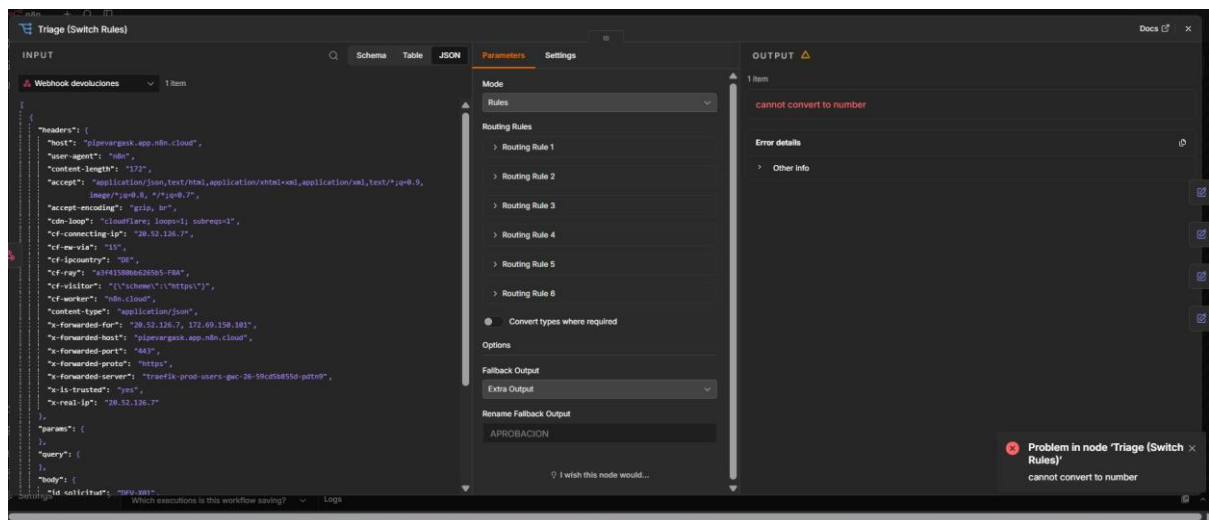


Figura 9 — Ejecución fallida de DEV-X01. El nodo Triage (Switch Rules) se detiene con el mensaje cannot convert to number.

2. Los tres valores de On Error

Valor de On Error	Qué hace	Dónde lo apliqué y por qué
Stop Workflow	Detiene la ejecución completa. La ejecución queda marcada en rojo en el historial.	En diez de los doce nodos del triage: webhook, Switch, los cuatro Edit Fields, Consolidar rutas, Registro, Guardar registro en planilla y Respond to Webhook. Todo lo que toca la decisión o la trazabilidad se detiene, porque una solicitud mal clasificada o sin registrar es peor que una que falla a la vista.
Continue (using regular output)	El flujo sigue y el item arrastra una clave error.	En Indicadores mindicador.cl y en Notificacion al cliente. Son servicios externos: su caída no debe impedir que la solicitud se clasifique y se registre. El campo monto_uf queda en null y se deja constancia de eso.
Continue (using error output)	El nodo gana una segunda salida por donde se desvían los items con problemas.	No lo use en este flujo. Solo tiene sentido cuando existe una rama que haga algo distinto con el error, y en este MVP no hay un destino alternativo para una solicitud que no se pudo clasificar.

El criterio general fue el siguiente: continuar solo cuando la falla no compromete ni la decisión ni el registro. La advertencia que hay que evitar es la de un flujo configurado para continuar en todos sus nodos, porque entonces termina siempre en verde, el cliente recibe la confirmación de que su solicitud fue procesada y el registro nunca se escribió porque el nodo de la planilla falló en silencio. Un flujo que nunca falla no es lo mismo que un flujo que funciona.

3. Qué es un Error Workflow

Un Error Workflow es un workflow aparte que comienza con un nodo Error Trigger y que se asigna a otro workflow en su configuración. Cuando una ejecución de ese otro workflow termina en fallo, n8n dispara automáticamente el Error Workflow y le entrega el detalle de lo ocurrido: el nombre del workflow afectado, el último nodo ejecutado, el mensaje de error y el identificador de la ejecución. Su única función es que alguien se entere.

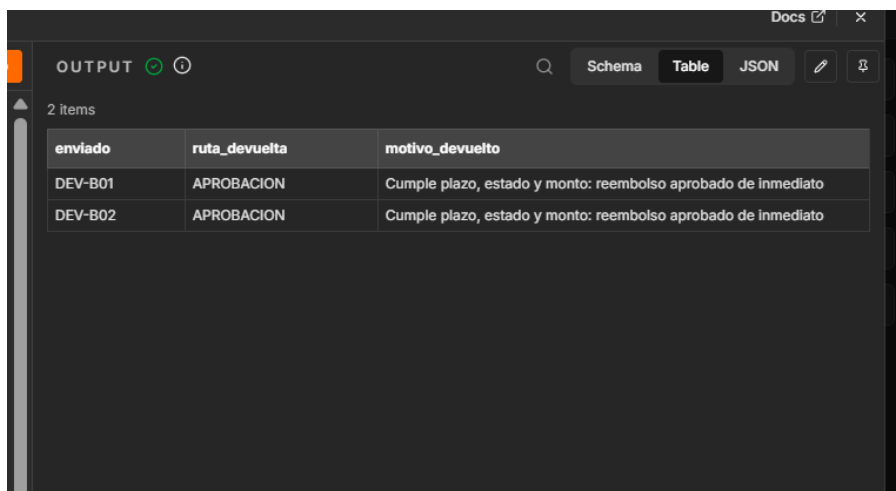
La diferencia con el On Error de un nodo es importante y conviene no confundirlas. El On Error decide que hace el flujo durante la ejecución con un ítem problemático; el Error Workflow se dispara después, cuando la ejecución completa ya fracasó, y no cambia nada de lo ocurrido.

Un flujo programado sin Error Workflow deja de funcionar en silencio, y esa es exactamente su situación más peligrosa. Un flujo disparado por webhook tiene a alguien esperando la respuesta: si falla, el cliente lo nota. Un flujo programado a las 20:00 no tiene a nadie mirando. Si falla, la ejecución queda en rojo en una lista de ejecuciones que nadie abre, no se envía el resumen, y al día siguiente la única señal disponible es la ausencia de un correo que uno podría atribuir a que simplemente no hubo movimientos. El problema puede sostenerse durante semanas antes de que alguien lo advierta.

4. Los límites: monto igual a U y días iguales a D

Las quince solicitudes entregadas no incluyen ningún caso de frontera, de modo que se construyeron dos adicionales. DEV-B01 se envió con monto exactamente igual a \$65.000 y DEV-B02 con exactamente 28 días desde la compra. Las dos quedaron clasificadas como APROBACION.

El resultado es el que exige la regla de comparaciones inclusivas y se explica por el operador utilizado. La regla del monto pregunta si el monto es mayor que 65.000, y 65.000 no es mayor que si mismo, de modo que la solicitud no entra a revisión y cae en el fallback. La regla del plazo pregunta si los días superan 28, y 28 tampoco supera a 28, de modo que la solicitud sigue estando en plazo. Si las condiciones se hubieran escrito con mayor o igual, ambas solicitudes habrían cambiado de ruta y el umbral y el plazo pasarían a comportarse como límites excluyentes, contradiciendo la política.



enviado	ruta_devuelta	motivo_devuelto
DEV-B01	APROBACION	Cumple plazo, estado y monto: reembolso aprobado de inmediato
DEV-B02	APROBACION	Cumple plazo, estado y monto: reembolso aprobado de inmediato

Figura 10 — Casos de frontera: monto igual a U y días iguales a D, ambos aprobados.

5. El error de tipo, a propósito

El enunciado pide cambiar el tipo de la condición del monto de Number a String y observar la ruta resultante. La versión de n8n utilizada no ofrece el operador mayor que para el tipo String, de modo que el experimento se realizó por la vía equivalente: se mantuvo el operador numérico y se retiró la conversión .toNumber() de la expresión, con lo cual el monto llega al operador como la cadena de texto que entregó el formulario.

El resultado observado no fue una clasificación errónea sino una falla: doce de las quince solicitudes terminaron con error 500 y solo tres completaron su ejecución. La explicación es que la validación de tipos del nodo es estricta y, al recibir una cadena donde espera un número, se niega a comparar y detiene el nodo antes de evaluar la condición. Las tres que sobrevivieron son DEV-1119, DEV-1123 y DEV-C03, y sobrevivieron porque salen por las reglas del plazo y del estado dañado, que se evalúan antes

que la del monto y por lo tanto nunca alcanzan la comparación defectuosa. Es una demostración no buscada de que el orden de las reglas determina incluso que solicitudes llegan a toparse con un defecto del flujo.

Conviene señalar el escenario que este experimento no produjo pero que estaba a un paso. Si la validación de tipos fuera laxa, o si se activara la opción *Convert types where required*, el nodo no fallaría: compararía texto con texto, carácter por carácter, de modo que la cadena "9990" resultaría mayor que "65000" porque el 9 viene después del 6, y DEV-C01, una solicitud de \$9.990, se derivaría a revisión manual. El flujo terminaría en verde, el cliente recibiría su respuesta y el registro quedaría escrito con la ruta equivocada. Ese es el error más peligroso de los dos, porque no deja ninguna señal. Tras la prueba la expresión se devolvió a su forma original y se verificó que las quince solicitudes volvieran a clasificarse correctamente.

15 items

enviado	×	ruta_devuelta	motivo_devuelto
DEV-1115		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1116		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1117		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1118		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1119		RECHAZO	Producto danado por uso
DEV-1120		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1121		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1122		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1123		RECHAZO	Producto danado por uso
DEV-1124		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1125		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-1126		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-C01		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-C02		SIN RESPUESTA - el triage fallo	500 - {"message": "Error in workflow"}
DEV-C03		RECHAZO	Fuera de plazo: 29 días desde la compra, el máximo D es 28

Figura 11 — Resultado tras retirar la conversión de tipo: doce solicitudes fallan y solo sobreviven las tres que salen por reglas anteriores a la del monto.

6. Los dos casos que el flujo no puede resolver

El primero es una solicitud con "monto": "veinte mil". Ocurre porque la regla 2 aplica `.toNumber()` sobre ese texto y no existe conversión posible: el nodo *Triage* (*Switch Rules*) falla con el mensaje *cannot convert to number* y el webhook responde con código 500. Para detectarla y no simplemente caerse, el flujo necesitaría una validación de forma previa al *Switch* que verifique que el campo *monto* contiene únicamente dígitos y, si no los contiene, lo derive a la ruta de datos inválidos con un motivo propio en lugar de dejar que el nodo se detenga. Dicho esto, que el flujo falle en vez de adivinar es preferible: un monto escrito en palabras no tiene interpretación única y cualquier conversión automática sería una suposición.

El segundo es una solicitud con "sku": "ZZ-999". Ocurre porque ese SKU no existe en el catálogo de productos y el flujo nunca consulta el catálogo: ninguna de las seis reglas mira el campo *sku*, de modo que la solicitud atraviesa toda la validación y se clasifica como *APROBACION* igual que cualquier otra legítima. Para detectarla el flujo necesitaría un nodo que cruce el *sku* contra el catálogo antes de clasificar y, ya que ese cruce se haría de todos modos, verificar también que el monto declarado coincida con el producto de unidades por el precio de lista. Sin esa validación, un cliente podría solicitar el

reembolso de un producto inexistente, o del precio equivocado de uno existente, y el proceso lo aprobaría sin dejar ninguna señal de alerta.

7. Las dos URL del webhook

n8n publica dos direcciones para un mismo webhook, que se diferencian únicamente en el segmento de la ruta. La URL de prueba, <https://pipevargask.app.n8n.cloud/webhook-test/devoluciones-135>, solo responde mientras el botón Listen for test event está activo y atiende una única llamada, tras la cual deja de escuchar; a cambio muestra los datos en vivo sobre el lienzo del editor. La URL de producción, <https://pipevargask.app.n8n.cloud/webhook/devoluciones-135>, responde siempre que el workflow este publicado y atiende llamadas sin límite, pero las ejecuciones no se dibujan en el editor y solo pueden consultarse después en la pestaña Executions, y únicamente si el workflow esta configurado para guardarlas.

Todas las evidencias de este informe corresponden a la URL de producción. Los propios items lo confirman: cada solicitud recibida trae los campos webhookUrl y executionMode, y este último registra el valor production.

Recomendación: el MVP debe convertirse en el sistema definitivo

La recomendación es adoptarlo, con condiciones. El argumento a favor es que el MVP resuelve el cuello de botella que motivo el proyecto y lo demostró sobre datos reales, no sobre una prueba de laboratorio. El triage, que hoy demora tres días hábiles frente a las veinticuatro horas prometidas, paso a resolverse en segundos, y un tercio de las solicitudes se cierra sin que nadie las toque. Las que sí requieren juicio humano llegan al equipo ya clasificadas y con el motivo escrito, de modo que el trabajo manual que queda es el que efectivamente lo necesita. A esto se suma que cada decisión queda registrada con su motivo y con los parámetros aplicados, algo que el proceso manual no producía en absoluto.

Las condiciones son tres y corresponden a brechas que las propias pruebas dejaron a la vista. La primera es la validación contra el catálogo: mientras el flujo no cruce el sku y el monto contra la lista de productos, seguirá aprobando solicitudes de artículos inexistentes sin señal alguna, y ese es un riesgo económico directo. La segunda es la capacidad: al enviar las quince solicitudes con cuatrocientos milisegundos de separación, cuatro superaron el tiempo de espera por congestión, y solo al espaciarlas a cuatro segundos el lote completo se procesó. Quince solicitudes al día no son un problema, pero un formulario web no las entrega espaciadas, y un día de alta demanda las entrega en ráfaga. La tercera es la visibilidad de la política: hoy las reglas viven dentro de un nodo que solo puede leer quien abre el editor de n8n, de modo que el área que define la política comercial no puede verificar ni modificar el umbral sin pasar por quien construyó el flujo.

Hay además dos limitaciones estructurales que no son defectos de la implementación sino de la herramienta, y que conviene dejar enunciadas. Una planilla de cálculo funciona como registro con quince filas diarias, pero no es una base de datos y deja de servir mucho antes de lo que uno supone. Y n8n no provee interfaz para el equipo que opera el proceso: quien revisa las solicitudes derivadas necesita un lugar donde trabajarlas, y ese lugar hoy no existe. Ninguna de las dos invalida la recomendación, porque n8n no deja de servir cuando el volumen crece, sino que deja de ser suficiente, y reconocer ese punto de antemano es parte de la decisión y no una objeción a ella.